

Hardware Implementation of Neural Network Inference Using SystemVerilog - Phase 4: Sequential Neural Network

1. Introduction

This phase continues our previous work on implementing a sequential neural network in SystemVerilog. This phase heavily relies on the previous phase, where our main objective was to design a combinational network with the same $2 \rightarrow 4 \rightarrow 2$ structure we have been discussing in Phase 2. In this final phase, we will implement the sequential version with the same fixed-point parameters, which demonstrates how real hardware, such as FPGA or ASIC, performs inference.

This phase consists of three main sections.

Section two will focus on implementing the Sequential Neural Network in SystemVerilog.

Section three focuses on verifying the network performance and further architectural improvements that can be applied.

Finally, in Section three, the trade-off between choosing the Combinational vs. Sequential architecture will be discussed.

2. Sequential Neural Network

The main objective of this section is to demonstrate the sequential architecture in the network and the idea of resource sharing.

2.1 Module Network

The module has a similar structure to the previous combinational one we've been designing in Phase 3. A parameterized network, whose parameters are the signal/register widths.

```
module network #(
    parameter int N_INPUTS = 2,
                N_NEURONS = 4,
                N_OUTPUTS = 2,
                DATA_WIDTH = 16,
                ACC_WIDTH = 48
```

An explanation regarding the widths is provided in the following.

2.2 Signal/Register Width

As shown above, the parameters N_INPUTS , $N_NEURONS$, $N_OUTPUTS$, and $DATA_WIDTH$ are almost identical to the combinational network.

The only difference lies in the width of ACC_WIDTH , where a wider width was selected to handle the accumulation during sequential computation.

Signal	Width	Type
x	16 bits	Input signal
$w^{(1)}$	16 bits	Input signal
$b^{(1)}$	16 bits	Input signal
acc	48 bits	Register
h	48 bits	Register
$w^{(2)}$	16 bits	Input signal
$b^{(2)}$	16 bits	Input signal
$z^{(2)}$	48 bits	Register

Table 1. Signal Width and Types

2.2.1 Design Trade-Off

acc is the register that keeps track of the value of the accumulator. It is worth mentioning that only one register was considered to handle the computation during cycles. The choice of the right width for the acc is a trade-off. One might consider declaring two registers to keep track of the separate accumulators in each layer, i.e., one 32-bit accumulator register for the hidden layer, and a 48-bit accumulator for the output layer. This, however, contradicts the idea of **resource sharing**, one of the main reasons for shifting from the combinational architecture to the sequential architecture.

This project specifically focuses on sharing one MAC block during several FSM states. Therefore, only one accumulator register was considered, although it needs more hardware. This way, the computation will be stored in a 48-bit register, regardless of the layer they are currently computing.

2.3 Ports

As discussed in Phase 1, the two important signals of a sequential MAC are `clk` and `reset`, which are clock signal and control signal, respectively. Another control signal that will be introduced in this section is called `start`.

```
input logic clk,  
input logic reset,  
input logic start,
```

The three above signals are control signals that are used for controlling the FSM states and computation.

The rest of the input ports are almost identical to the combinational design:

```
input logic signed [DATA_WIDTH-1:0] x[N_INPUTS],  
input logic signed [DATA_WIDTH-1:0] w1[N_NEURONS][N_INPUTS],  
input logic signed [DATA_WIDTH-1:0] b1[N_NEURONS],  
  
input logic signed [DATA_WIDTH-1:0] w2[N_OUTPUTS][N_NEURONS],  
input logic signed [DATA_WIDTH-1:0] b2[N_OUTPUTS],  
  
output logic signed [ACC_WIDTH-1:0] z2[N_OUTPUTS],  
output logic pred_class,  
output logic done
```

`done` is a control signal that will be asserted when the prediction is made by the last FSM state.

2.4 FSM States

So far, the term “FSM state” has been applied without providing an exact definition of what an FSM is.

Finite State Machines (FSMs) are the fundamental building blocks of sequential digital systems, describing how systems transition between distinct states based on inputs. FSMs provide a systematic framework for designing predictable, reliable sequential behavior in every digital system.

They are simply a way of describing how something behaves by breaking it down into distinct "states" or conditions. States can be considered as different modes or situations that something can be in. The word "finite" just means there's a limited, countable number of these states—not infinite. *Adapted from [1]*

The key idea is that at any moment, the system is in exactly one state. [1]

The states of the FSM and their behaviors are shown in the following table:

FSM State	Behavior
IDLE	Waits for the start signal
HIDDEN_MAC	Computes hidden layer's MAC operation
HIDDEN_STORE	• Adds the Bias + ReLU • Also checks for more hidden neurons ◦ If more hidden neurons exist: ■ Transits the state to HIDDEN_MAC for further computation ◦ Otherwise, transits the state to OUTPUT_MAC
OUTPUT_MAC	Computes output layer's MAC operation
OUTPUT_STORE	• Adds Bias • Checks for more output neurons ◦ If more output neurons exist: ■ Transits the state to OUTPUT_MAC for further neuron computation ◦ Otherwise, transits the state to DONE
DONE	• Applies Argmax and makes prediction • Asserts done • Transits to IDLE

Table 2. FSM States

2.4.1 FSM SystemVerilog Implementation

To define the FSM states, a new data type called ``state_t`` was declared that stores the FSM states. This was done using `typedef` and `enum`.

`enum` was applied to handle the 6 FSM states and to avoid hard-coding the states' indices.

`typedef` was utilized to declare a new data type.

In complex testbenches, some variable declarations might have a longer data-type specification or require being used in multiple places in the testbench. In such cases, we can use a `typedef` to give a user-defined name to an existing data type. The new data type can then be used throughout the code and hence avoids the need to edit in multiple places if required. [2]

typedef and enum can be used together to make it easier to declare variables of the enum type, without having to write enum every time.

In code, the syntax is as follows: [2]

```
typedef enum logic [2:0] { ... } state_t;
```

In this work:

```
typedef enum logic [2:0] {  
  
    IDLE,  
    HIDDEN_MAC,  
    HIDDEN_STORE,  
    OUTPUT_MAC,  
    OUTPUT_STORE,  
    DONE  
  
} state_t;  
  
// FSM states  
state_t state;
```

The total number of states in this work is 6, as mentioned above. Note that the range of the states must be a power of 2. The smallest integer that can store 6 states is 3. Using 3 bits, there are 8 possible states, while the FSM will only occupy 6 states.

2.5 Internal Registers

Other internal registers were declared as follows:

2.5.1 Accumulator

The accumulator was declared as follows, with a width of 48 bits:

```
logic signed [ACC_WIDTH-1:0] acc;
```

2.5.2 Hidden Layer Activations

The activations, i.e., outputs of the ReLU, were stored in an array with 4 elements:

```
logic signed [ACC_WIDTH-1:0] h[N_NEURONS];
```

2.5.3 Counters

An important design architecture is the use of counters, where one is the index of the current neuron in the hidden/output layer, i.e., the n-th neuron, and another that controls the index of the current input, i.e., the i-th input.

```
logic [1:0] neuron_idx;  
logic [1:0] input_idx;
```

The number of bits required for each counter is determined by the maximum number of neurons and inputs, respectively.

The hidden layer has 4 neurons, and the output layer contains 2 neurons. The maximum number of neurons in this case is 4, which can be stored in a 2-bit register (which has 4 states in total).

The number of inputs at first glance is 2, i.e., 2 features, whereas the number of inputs of the output layer, i.e., activations of the hidden layer, is 4. Therefore, a 2-bit register was declared for storing the 4 inputs.

Note that for this work, a 2-bit register was declared. Though it is worth mentioning, to improve this version of the code further, the counters could also be declared parameterized. A fully parameterized version would instead derive the width from the parameters (using `$clog2`), so the module scales automatically.

2.6 FSM-Controlled Sequential Datapath

The following SystemVerilog code blocks demonstrate the Sequential Neural Network Controller.

2.6.1 Asserted reset

```
always_ff @(posedge clk) begin : seq_ann  
  
    // reset all registers  
    if (reset) begin
```

```

state <= IDLE;
neuron_idx <= 0;
input_idx <= 0;
acc <= 0;
done <= 0;
pred_class <= 0;

for (int i = 0; i < N_NEURONS; i++) begin

    h[i] <= 0;

end

for (int i = 0; i < N_OUTPUTS; i++) begin

    z2[i] <= 0;

end

end

```

The above blocks of code demonstrate that when the signal `reset` is asserted, every register will be reset. It sets the FSM to `IDLE`, clears the accumulator (`acc`), resets the counters (`neuron_idx`, `input_idx`), clears `h[ ]` and `z2[ ]`, deasserts `done`, and clears `pred_class`. Its purpose is to initialize the FSM and all registers to a known state before inference begins.

2.6.2 Deasserted reset

If the `reset` is deasserted, inference begins to be computed (i.e., FSM state transitions).

2.6.2.1 IDLE

```

case(state)

IDLE : begin

    done <= 0;

    if (start) begin

```

```

        state <= HIDDEN_MAC;
        neuron_idx <= 0;
        input_idx <= 0;
        acc <= 0;

    end
end

```

The transition of states is done inside “switch cases”. The first case is IDLE. The **primary role** of IDLE is to **wait for the start signal**. While doing so, it also prepares the registers that must be initialized before a new inference begins.

2.6.2.2 HIDDEN_MAC

```

HIDDEN_MAC : begin

    acc <= acc + ((w1[neuron_idx][input_idx] * x[input_idx]) >>> SHIFT_FACTOR);

    if (input_idx == (N_INPUTS - 1)) begin

        state <= HIDDEN_STORE;

    end

    else begin

        input_idx <= input_idx + 1;
        state <= HIDDEN_MAC;

    end

end
end

```

The hidden neuron is computed in the HIDDEN_MAC state. This state is responsible for computing the weighted sum of a neuron and does this process until there are no more inputs. When all inputs have been involved in the computation, and no more inputs are left, it transitions to the next state, i.e., HIDDEN_STORE.

2.6.2.3 HIDDEN_STORE

```
HIDDEN_STORE : begin

    h[neuron_idx] <= ReLU(acc +
{{(ACC_WIDTH-DATA_WIDTH){b1[neuron_idx][DATA_WIDTH-1]}}, b1[neuron_idx]});

    if (neuron_idx == (N_NEURONS - 1)) begin

        input_idx <= 0;
        neuron_idx <= 0;
        acc <= 0;
        state <= OUTPUT_MAC;

    end

    else begin

        neuron_idx <= neuron_idx + 1;
        state <= HIDDEN_MAC;
        input_idx <= 0;
        acc <= 0;

    end

end
```

HIDDEN_STORE has two important behaviors, as explained in Table 2. This state is responsible for adding the sign-extended bias to the acc and applying ReLU to compute the activations. It also checks whether there are more neurons left. If so, it will change the state to HIDDEN_MAC to redo the computation of the next neuron in the hidden layer. Otherwise, if all activations have been computed and stored in the array `h[]`, it will transition to the next state, OUTPUT_MAC.

2.6.2.4 OUTPUT_MAC

```
OUTPUT_MAC : begin

    acc <= acc + ((w2[neuron_idx][input_idx] * h[input_idx]) >>> SHIFT_FACTOR);

    if (input_idx == (N_NEURONS - 1)) begin
```

```

        state <= OUTPUT_STORE;

    end

    else begin

        input_idx <= input_idx + 1;
        state <= OUTPUT_MAC;

    end

end
end

```

The architecture of the OUTPUT_MAC is identical to the HIDDEN_MAC. The difference lies in the input of this layer, i.e., the output layer, which is the array of activations.

A key idea to take into consideration is that the multiplication of the 48-bit activations and 16-bit weight matrix will result in a 64-bit accumulator. However, since the width of the accumulator is 48 bits, it will be truncated to the original width.

This truncation has not been effective in the produced results. However, the right choice and the trade-off should be considered in further FPGA projects, as discussed in Section [2.2.1](#).

2.6.2.5 OUTPUT_STORE

```

OUTPUT_STORE : begin

    z2[neuron_idx] <= acc + {{(ACC_WIDTH-DATA_WIDTH){b2[neuron_idx][DATA_WIDTH-1]}},
b2[neuron_idx]}};

    if (neuron_idx == (N_OUTPUTS - 1)) begin

        state <= DONE;

    end

    else begin

        neuron_idx <= neuron_idx + 1;
        input_idx <= 0;
        acc <= 0;
    end
end

```

```

        state <= OUTPUT_MAC;

    end
end

```

The OUTPUT_STORE behaves similarly to the HIDDEN_STORE. This state, however, won't pass the logits to any decision function. It merely adds the sign-extended bias to the accumulator. If any other neurons have been left in the output layer, it passes them to the OUTPUT_MAC to compute the remaining neurons using the activations. Otherwise, the state will be changed to DONE.

2.6.2.6 DONE

```

    DONE : begin

        // prediction
        pred_class <= Argmax(z2[0], z2[1]);
        done <= 1;
        state <= IDLE;

    end

endcase
end
end

```

Finally, in the state DONE, the predictions are made by Argmax. Also, the done register is asserted, and the state will be transitioned to IDLE.

2.6.3 Decision Functions

The decision functions, i.e., ReLU and Argmax, were written as functions wrapped up in a package instead of modules. This helps to simplify the logic behind the code in the sequential design, where each signal and register is updated on positive edge clocks.

To wrap the functions in a package, a similar approach to Java is applied as shown below:

```

package decision_functions;

```

2.6.3.1 ReLU

Implementing ReLU is as follows:

```
function automatic logic [47:0] ReLU(  
  
    input logic signed [47:0] in  
);  
  
    return (in < 0) ? 0 : in;  
  
endfunction
```

As shown above, the architecture of ReLU follows a combinational design. The condition is applied to the input using a Ternary Operator.

2.6.3.2 Argmax

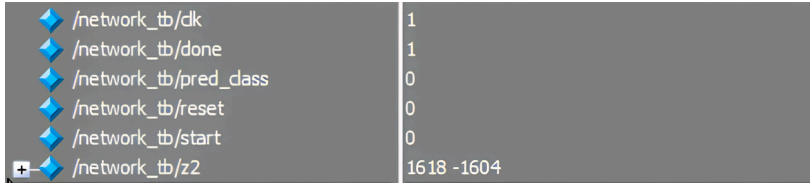
```
function automatic logic Argmax(  
  
    input logic signed [47:0] o0,  
    input logic signed [47:0] o1  
);  
  
    return (o0 >= o1) ? 0 : 1;  
  
endfunction  
  
endpackage
```

3. Verification

The quantized test set's features and the trained parameters retrieved from Python have been used for testing the Network module performance in QuestaSim.

The key idea here is that the quantized `X_test`, weights, and biases were all derived from the testbench, not from a separate parameters ROM, i.e., Weights ROM and Bias ROM. This option is worth experimenting with in future work.

A waveform of a sample with X_test values of [980 -1584] is provided below:



/network_tb/dk	1
/network_tb/done	1
/network_tb/pred_class	0
/network_tb/reset	0
/network_tb/start	0
/network_tb/z2	1618 -1604

Figure 1. Network Prediction of the Last Sample

In QuestaSim Transcript:

```
# Sample 56:
# x = [980, -1584]
# h = '{0, 2327, 0, 2211}'
# z2 = '{1618, -1604}'
# Prediction = 0
# Ground Truth: = 1
```

The testbench also computed the accuracy at the end of the inferences to verify the network performance in hardware:

```
# correct predictions: 53
# incorrect predictions: 4
# accuracy: 92.98
```

3.1 Network Architecture Analysis

During verification, a considerable number of test samples produced zero activations for all hidden neurons, i.e., h0 and h3 after the ReLU layer. Consequently, the final prediction for these samples depended solely on the output-layer biases. This behavior suggests that the hidden layer contains inactive neurons for a subset of the input distribution. Future work could investigate whether a smaller hidden layer (e.g., 2→2→2) maintains comparable accuracy, or whether alternative activation functions such as Leaky ReLU reduce the number of inactive neurons. Such modifications would require retraining and quantitative comparison rather than being inferred solely from the observed activations.

4. Combinational vs. Sequential Trade-Off

This section focuses on a comparison between running the inferences using a Combinational vs. a Sequential network. Also, the number of cycles (latency) it takes for one inference will be counted.

Combinational	Sequential
Single-cycle computation	Multi-cycle computation
More hardware	Less hardware
More multipliers/adders/registers	Shared MAC
Faster	Slower
More parallelism	More reuse
Higher area	Lower area

Table 3. Combinational vs. Sequential trade-off

In conclusion:

- **Combinational:** prioritizes **speed** at the cost of **hardware resources**.
- **Sequential:** prioritizes **resource efficiency** at the cost of **execution time**.

4.1 Latency

This final part focuses on counting the **latency**, which is the number of clock cycles from the start of an inference until the prediction is valid. In this work, it starts counting at the clock edge where the FSM leaves IDLE (i.e., when start is sampled), and stops at the clock edge where done is asserted and pred_class becomes valid.

Layer	Computation	Clock Cycles
Hidden Neuron	2 MAC + 1 Store (Bias + ReLU)	3
Hidden Layer	4×3	12
Output Neuron	4 MAC + 1 Store (Bias)	4
Output Layer	2×5	10
DONE	Argmax, Assert done	1
Total	12 + 10 + 1	23

Table 4. Sequential Network Latency

References

- [1] "Finite State Machines," ChipVerify. [Online]. Available:
<https://chipverify.com/digital-fundamentals/finite-state-machines>. [Accessed: Jul. 21, 2026].
- [2] "SystemVerilog typedef and alias," ChipVerify. [Online]. Available:
<https://chipverify.com/systemverilog/systemverilog-typedef-alias>. [Accessed: Jul. 21, 2026].
- [3] "SystemVerilog Functions," ChipVerify. [Online]. Available:
<https://chipverify.com/systemverilog/systemverilog-functions>. [Accessed: Jul. 21, 2026].